



Regression Approach to Predicting Mobile Device Battery Health

A data-driven framework for spoof-resistant battery health verification

Battery Health Predictor

Battery Health Predictor

TECNO BC1 · Android 10

⚡ Connect Device

DEVICE TELEMETRY

⚡ Voltage

4.350 V

ADB

🌡️ Temperature

32.0 °C

ADB

📱 Design Cap

needs input

manual required

🔋 Charge Cycles

from days

estimated

🔋 Charge Level

91 %

ADB

🔄 Status

Charging

ADB

💖 Health Tag

Good

ADB

🔌 Power Source

unavailable

not exposed

📱 Android Ver

10

ADB

⚠️ Enter manually: factory_capacity, charge_cycles

⚡ Voltage (V)

4.350

🌡️ Temperature (°C)

30.0

📱 Design Cap (mAh)

4500

📅 Days Used (cycles estimated as days × 1.25)

1500

☒ Estimate charge cycles from days used instead

▶ Predict Battery Health

PREDICTED HEALTH

88.3%

☒ Good – battery is healthy.

Predicted SOH

652.8 mAh

Train Scale

739.1 mAh

Device Cap

4500 mAh

TELEMETRY LOG

Inference Input Log

Device	TECNO BC1
Voltage	4.350 V [ADB]
Temperature	32.0 °C [ADB]
Factory Cap	4500 mAh [MANUAL]
Charge Cycles	1875 [MANUAL]

Predicted SOH

652.80 mAh

Training Scale

739.11 mAh

Health %

88.32%

⚠️ Health uses training-scale denominator (739.1 mAh), not device factory cap.



Agenda

1. Introduction and Problem Statement
2. Battery Health Concepts and Data Preparation
3. Modeling and Validation
4. Deployment and Future Work



Introduction

- **The problem**

- As market of **used mobile devices** increases, we noticed that too many users complain about **battery performance that doesn't match seller specifications**.
- Sellers may use **spoofing techniques** to hide the actual battery health of a device or just say it out verbally which leads to **unfair pricing and loss of trust** in this type of market that affects fair traders.
- This is because battery health degrades overtime and this is a huge turnoff for consumers which forces sellers to fake percentages to make more sales.

The Solution

- Our machine learning model utilizes information read directly from the hardware of the phone, bypassing any kernel compromises and using it to predict the State Of Health (SOH Capacity) of the battery.





Why This Matters

The visible problem

- The settings page can no longer be trusted for true battery health representation.
- Batteries in poor health have tendencies to **heat up, drain fast and waste power.**
- If severe enough old batteries become **fire hazards due to internal short circuits and higher internal resistance**

Global use potential

- As edge computing and AI become mobile, battery health will soon become very important and this model acts as a baseline for future models to perform even better predictions of **battery degradation overtime, battery failure predictions and much more.**





Our Objectives



Variable identification

- To identify and assess which variables and how much they affect the performance and health of a battery overtime.



Model development

- To develop, train and test a ML model to use device hardware telemetry to predict battery health.



Prototyping

- To develop a web, desktop or mobile interface for easy access of the model by end users.



How it works

SOH Capacity

- We define **State of Health (SOH)** as the maximum capacity the battery can hold at a given time.
- This is what actually determines how much power the battery holds per full charge hence its health.
- **Battery health = measured SOH / factory SOH * 100 %**

Why we target SOH

- We use SOH as the single target our regression models learn to predict.
- This is because this cannot be measured directly by a phone hence is not available in the kernel.
- We instead use other metrics like **charge cycles, voltage, temperature and factory capacity. which are all accessible via USB.**





Dataset choice

Data foundation

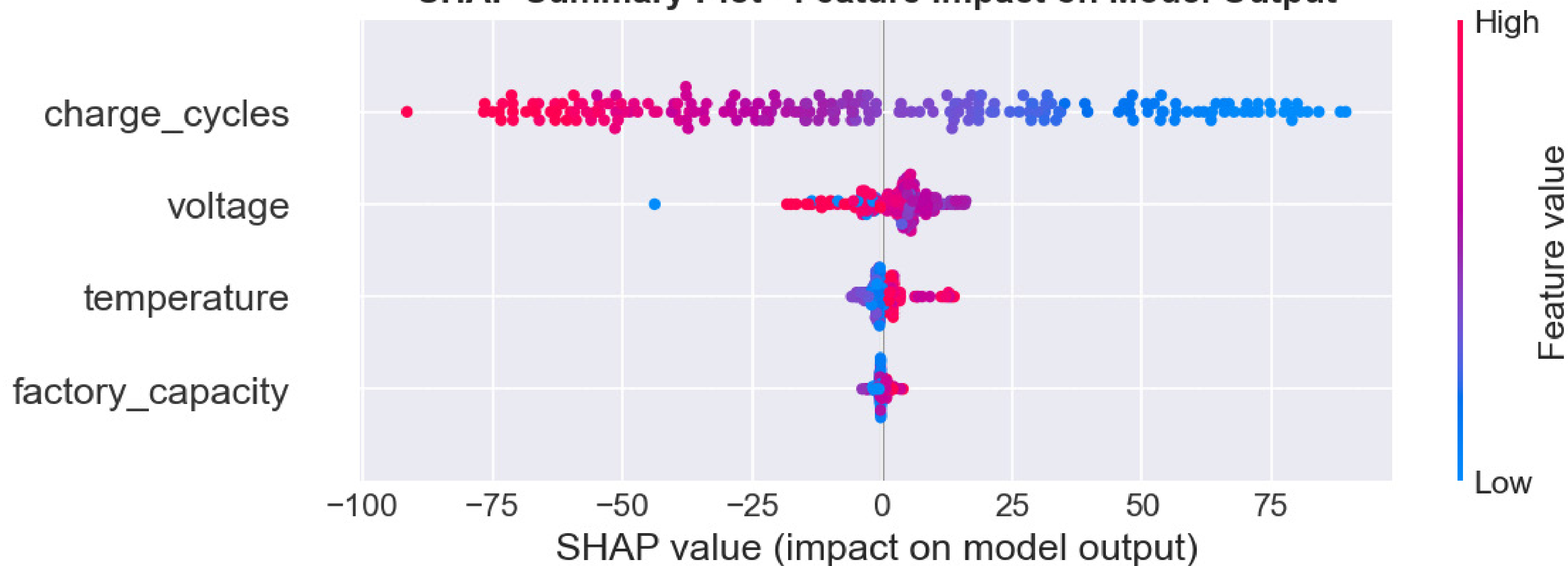
- We used the **Oxford Battery Dataset** to capture long-term degradation behavior.
- This was collected by monitoring the Li-ion battery metrics of several cells overtime in several charge cycles and their performance in several temperatures measured together with their SOH totalling 519 entries.

Key features of dataset

- **Charge cycles:** Total number of times the battery has been charged and discharged.
- **Average Voltage:** Voltage of the battery at time of measurement.
- **Average Temperature:** Heat given off by battery at that time.
- **Max Factory SOH:** The capacity of the battery out of the box.
- Other fields like **Cell ID** are ignored due to irrelevance.



SHAP Summary Plot - Feature Impact on Model Output





EDA and pre-processing



Feature selection

- We dropped **days-used-so-far** since it was engineered at conversion of the dataset from **mat format to csv** and found to contain noise.
- We also tried multiple domain specific engineered features but they had little to no effect on the model performance.



Scaling and Standardization

- We used a **standard scaler** to ensure **scale** large features like charge cycles that are large and skew the model, and also to **standardize** the features.
- This significantly improved model performance.



EDA results

- Most features were **normally distributed** apart from **charge cycles and factory SOH**.
- **Charge cycles** showed the highest correlation with the target of over **-0.95** and the rest ranging between **-0.03 and 0.18**.



Model Selection

What we considered

- We trained **6 separate models** on the same data using **GridSearchCV** and **K-Fold cross validation** to ensure we got the best picture and chose the absolute best model for the regression task.
- We looked for lower **MAE** to reduce practical prediction error, We prioritized **R-squared** to confirm overall fit to degradation trends. The results were as follows.

===== FINAL MODEL RANKINGS =====				
	Algorithm	MAE	R2	Score
9	XGBoost (Tuned)	117.548	0.6950	
8	LightGBM (Tuned)	118.703	0.6889	
11	XGBoost (Boosted)	118.211	0.6882	
10	LightGBM (Boosted)	118.134	0.6873	
2	Random Forest	119.906	0.6717	
6	Random Forest (CV mean)	122.697	0.6460	
0	Linear Regression	129.121	0.6181	
4	Linear Regression (CV mean)	130.622	0.5884	
3	KNN Regressor	153.573	0.4420	
1	Decision Tree	156.398	0.3715	
7	KNN Regressor (CV mean)	160.623	0.3533	
5	Decision Tree (CV mean)	158.906	0.3400	

Final conclusion

- Based on the results **XGBoost** had the best overall performance.
- This is because it involved **inbuilt regularization** and **Gradient Descent** for optimization plus the benefit of **averaging** from several models.



Model evaluation

Cross-validation

- We used K-fold cross-validation to test performance across different splits.

GridSearch CV

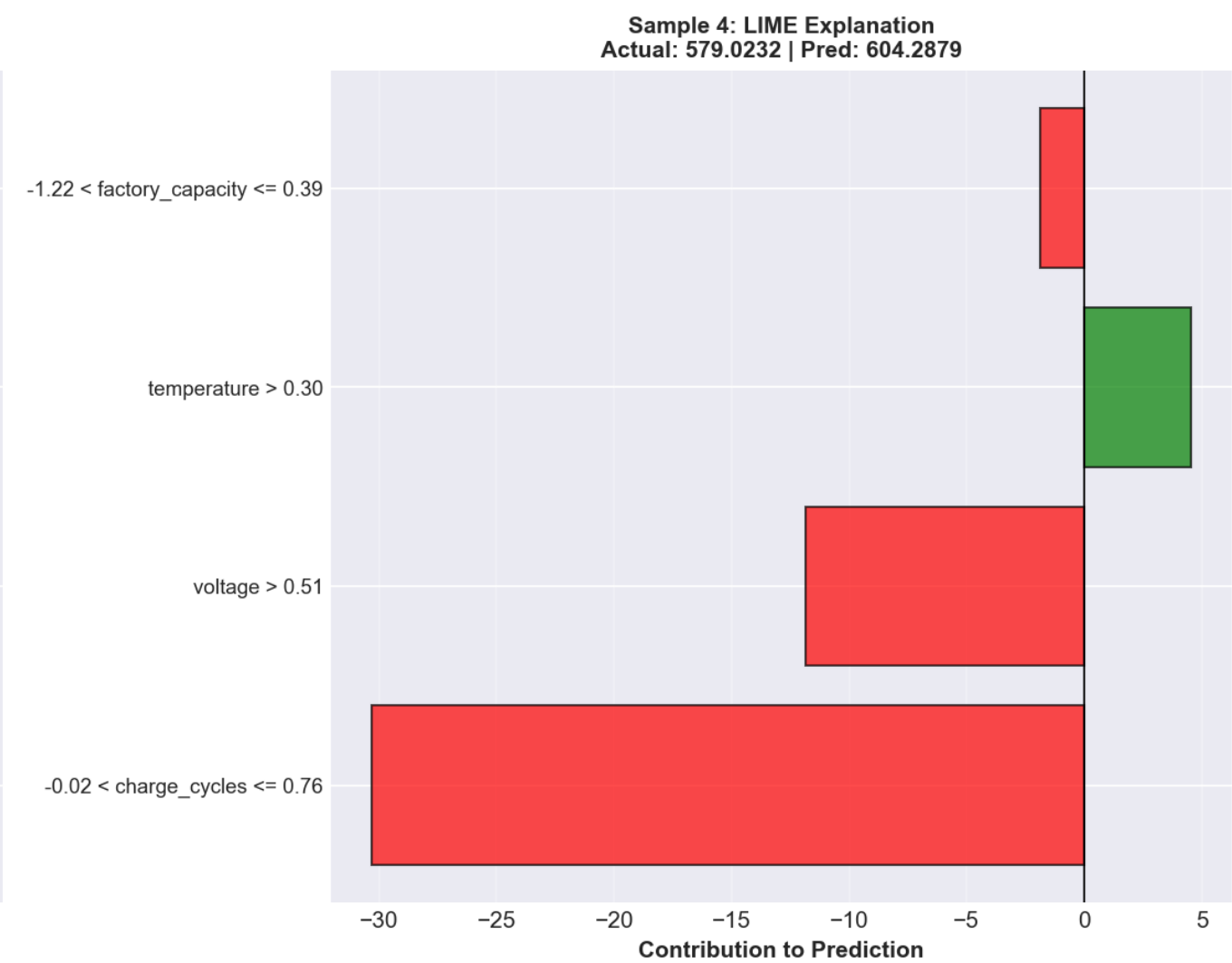
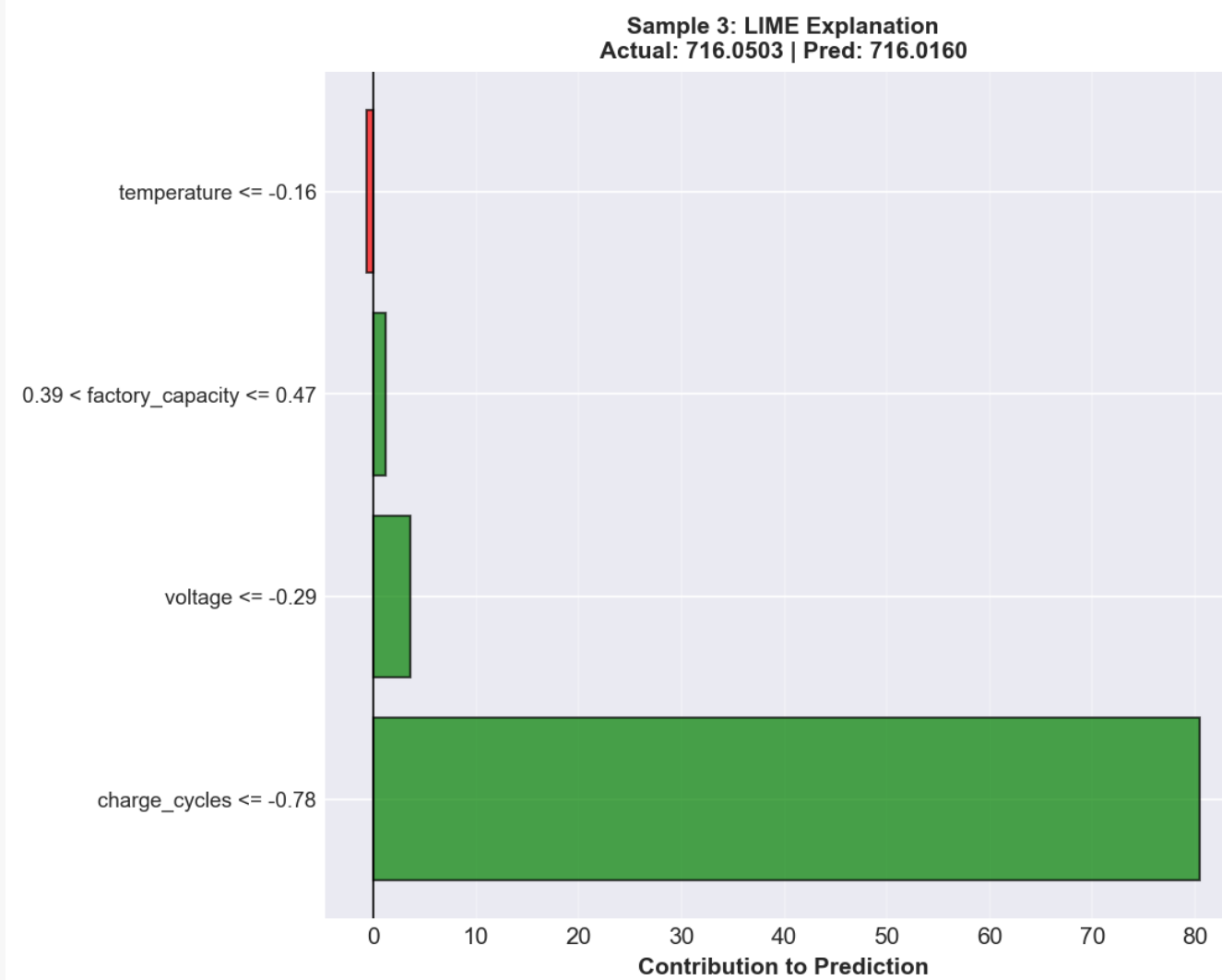
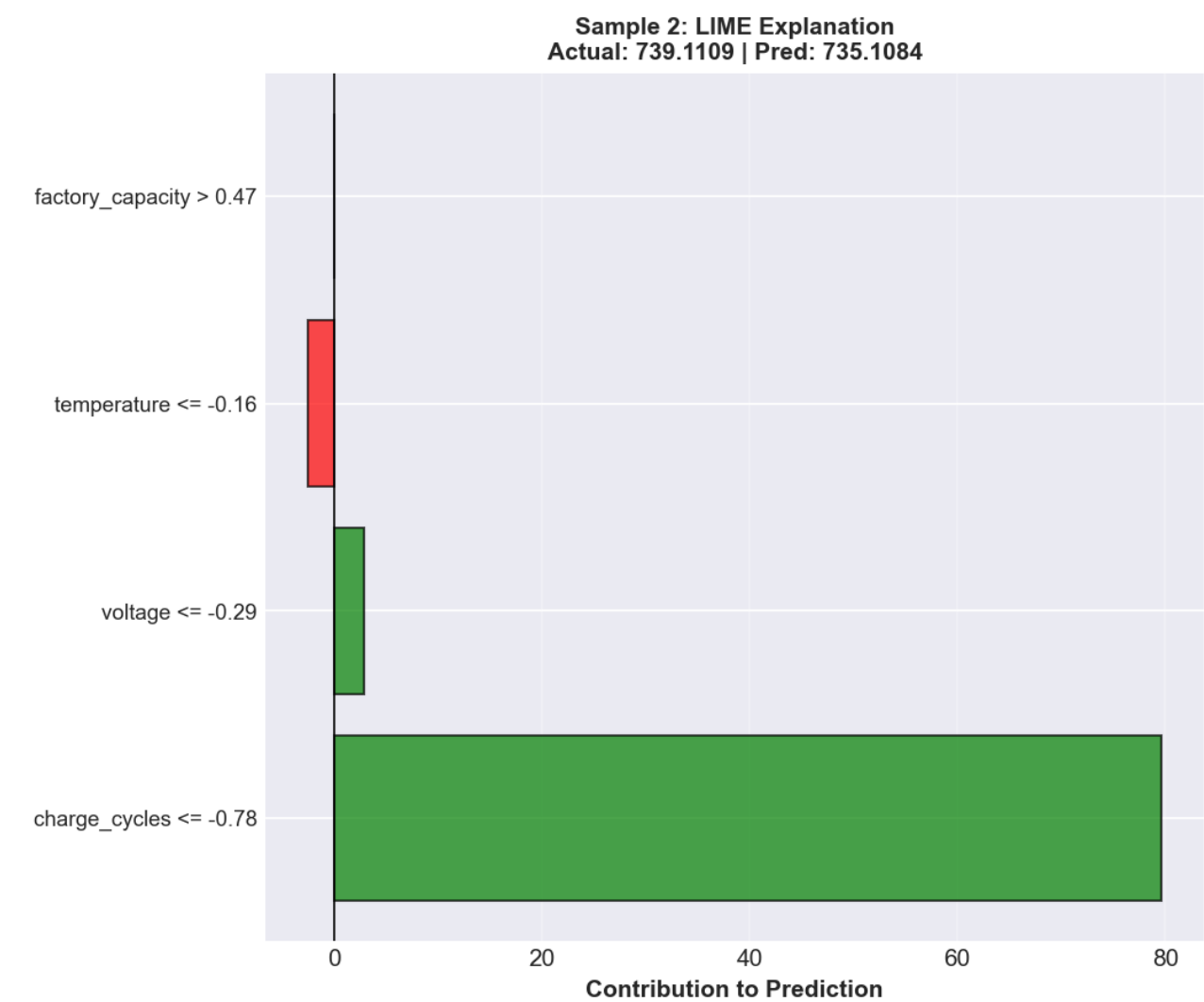
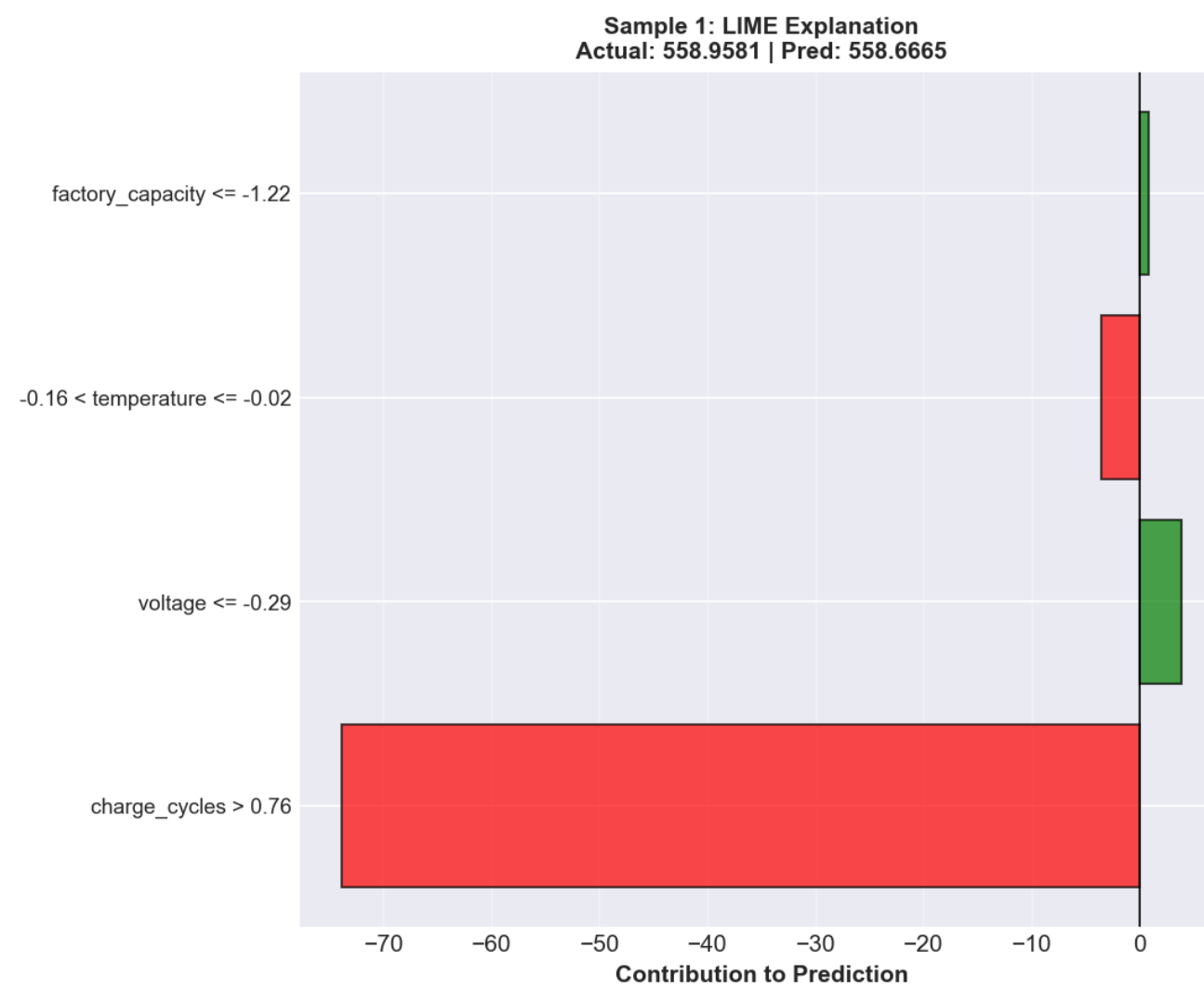
- We selected settings that improved MAE and maintained strong R-squared from grid search.

Pipelines

- Since pipelines were used, we are confident that there was no data leakage

```
--- Training Gradient Boosting Regressor ---  
Cross-val MAE: 2.9932 +/- 0.8789
```

```
--- Test Set Performance ---  
MAE: 1.8942 (capacity units)  
MSE: 11.705865  
RMSE: 3.4214  
R2: 0.9959  
MAPE: 0.30%
```





Problems encountered

01

Dataset selection

- Most datasets available online used **hardware specifications** to determine the battery health by predicting **number of days used** which was innacurate.
- Other datasets were **tailored for other devices** not phones hence rendering their data irrelevant.

02

Feature selection and engineering

- The dataset like **SOH capacity percentage** and **days used so far** that caused overfitting.
- **Charge cycles** showed domination over most predictions but there is a proper scientific explanation for that.

03

Long training times

- **GridSearchCV** is exponentially slow depending on the number of hyperparameters given and number of models.
- We solved this by using **Random Search CV** instead which is a less accurate but more optimal CV estimator.



Future prospects



Predicting days remaining

- The model could be further trained to predict **how long a battery will last** given on previous conditions especially for IoT and Edge AI mobile devices in remote areas.



Battery fire warning

- Batteries in **electronic cars and mobile phones** can be connected to our simple model to **predict fires** based on changes in their **environment and internal state**.



Dynamic pricing

- The model could be expanded to consider the **overall specifications** of the phone or mobile device and **suggest a price**.
- This can help **guide consumers buying used gadgets**.



Model testing pipeline

Requirements

- We need a **USB cable** able to connect the devices below.
- A **computer or another phone (if available)** to run the model on .
- The **model software installed** on the chosen computer or smartphone.
- The **phone to be tested**.

Telemetry extraction

- The **mobile or desktop app** installed uses **ADB for Android** or **XCode for IOS** to collect the features from the phone's hardware.
- It the **pre-processes** it and passes it to the **joblib model** to make the prediction which is then **formatted and displayed** for the user on the app.

Some challenges

- **Not all devices** expose this information via USB.
- **USB debugging** must be enabled for the app to access the hardware.